# ROS 2 Search and Rescue Robot Project

## Scenario

A **natural disaster has destroyed a building**, and survivors may be trapped inside.

You must design a **mobile robot** capable of exploring the environment and locating important markers that represent hazards, survivors, and resources.

The robot will operate in a **Gazebo simulation environment** and must use **ROS 2** and a **Behavior Tree architecture** to complete its tasks.

The robot must:

- Spawn in a **Gazebo simulation world**
- Move using **ROS 2 Control**
- **Perceive objects** in the environment using sensors
- Use a **Behavior Tree** to manage search and rescue tasks
- **Locate specific markers** and complete tasks associated with them

The overall goal is to **identify objects, perform the required actions, and report important locations**.

# Environment Objects (Already in the Simulation World)

Each object in the world represents something the robot must interact with.

| Object | Meaning |
|---|---|
| Green Cuboid | Survivor |
| Blue Cube | Water dam |
| Red Cylinder | Fire / hazard |
| Yellow Cube | Medical kit |
| Purple Cube | Exit location |

■ Black square on wall Robot docking station

# Part 1 — Robot Modelling (5 marks)

You must **design your own robot model using URDF**.

Requirements:

- The robot design is completely up to you
- You **may not download a full robot model from the internet**
- You **may use tutorials** to help build your robot
- If your robot appears copied, you may be **interviewed to explain your design**

Your robot should include:

- A base
- Wheels or another locomotion system
- Mounts for sensors

# Part 2 — Spawn Robot in Gazebo (5 marks)

You must spawn your robot into the provided **Gazebo simulation world**.

Requirements:

- Use a **ROS 2 launch file**
- Use **robot_state_publisher**
- Spawn the robot at: x=0 and y=0

# Part 3 — Gazebo Plugins and ROS Integration (5 marks)

You must integrate sensors and physics using **Gazebo plugins**.

Minimum recommended sensors:

- **Camera**
- **Either:**
    - 2D LiDAR
    - Depth camera

Additional sensors strongly recommended:

- **NavSat (GPS)** or other localization aid
- **IMU**

These will help with **navigation and localization**.

# Part 4 — ROS 2 Control (5 marks)

You must integrate **ROS 2 Control** to allow your robot to move.

Your robot must be able to:

- Accept **velocity commands**
- Drive around the environment
- Be controlled by your ROS architecture

# Part 5 — Behaviour Tree and ROS Architecture (70 marks)

This is the **main part of the project**.

You must design a **ROS 2 architecture** and **Behavior Tree system** that allows the robot to complete the rescue tasks.

## ROS Architecture (7.5 marks)

Marks will be awarded for good ROS design practices, including:

- Well-structured **nodes**
- Proper use of **parameters**
- Use of **topics**
- Use of **services**
- Use of **actions**
- Optional use of **custom messages**

Your system should clearly separate responsibilities such as:

- Navigation
- Perception
- Task control
- Robot status

### Behaviour Tree Design (7.5 marks)

Your Behavior Tree must include:

- **Sequence nodes**
- **Fallback nodes**
- At least **one asynchronous stateful Action node**
- At least **one decorator node**

Your Behavior Tree should control the **entire mission flow**.

## Robot Tasks

Your robot must complete the following tasks.

# Task 1 — Bring Medical Kit to Survivor (10 marks)

1. Navigate to the **survivor (green cuboid)**
2. Stop **1 metre away** from the survivor
3. Wait **1 second**
4. Publish the **TF of the survivor relative to the spawn location**

5. Navigate to the **medical kit (yellow cube)**
6. Return to the **survivor**

# Task 2 — Scan the Dam for Damage (10 marks)

1. Navigate to the **dam (blue cube)**
2. Stop **50 cm away**
3. Aim the camera at the **centre of the dam**
4. Rotate the camera **10° left**
5. Rotate the camera **10° right**

You may either:

a. Rotate the **robot**, or
b. Use a **moving camera joint**

# Task 3 — Low Battery Behaviour (10 marks)

If the topic: battery_level_low publishes true

The robot must:

1. Find the **black docking square**
2. Stop at the docking station
3. Wait until the topic publishes: false

# Task 4— Fire Safety constraint (5 marks)

At all times the robot must stay at least: 3 meters away from the fire

# Task 5— Exit the Building (5 marks)

Once all tasks are completed:

1. Navigate to the **exit (purple cube)**
2. Touch the exit marker
3. Stop and wait

# Navigation to Objects (10 marks)

Your robot must be able to **navigate to the objects in the environment**.

You may use **any approach**, such as:

- **Nav2 navigation stack**
- A **search algorithm**
- A **custom navigation system**
- A **predefined waypoint system**

If your robot cannot autonomously reach the objects, it'lldrive it to the locations before running your Behavior Tree.

# Report (15 marks)

You must submit a **short report** explaining your system.

## Section 1 — ROS Architecture

Explain your **RQT graph**:

- What each node does
- Communication between nodes ie Topics and message types
- Services used
- Actions used

For each **ROS action**, explain:

- Goal
- Feedback
- Result condition

## Section 2 — Behaviour Tree

Explain your **Behavior Tree design**:

- What each node does
- Node inputs/outputs Ports used

## Section 3 — Navigation Strategy

Explain **how your robot moves to each object**.

For example:

- Nav2
- Custom search
- Waypoints
- Perception-based navigation

## Section 4 — References

Provide links to **all resources used**, including:

- Tutorials
- Documentation
- GitHub repositories
- Online articles

Failure to include resources that you have used may result in **interview questions** about your implementation. IE if you used a existing git repo URDF or an online tutorial and don’t link it. I don't mind the use of them once you understand them and don't just copy and paste. I will also interview you if it looks like you just copy and pasted.